Day 1

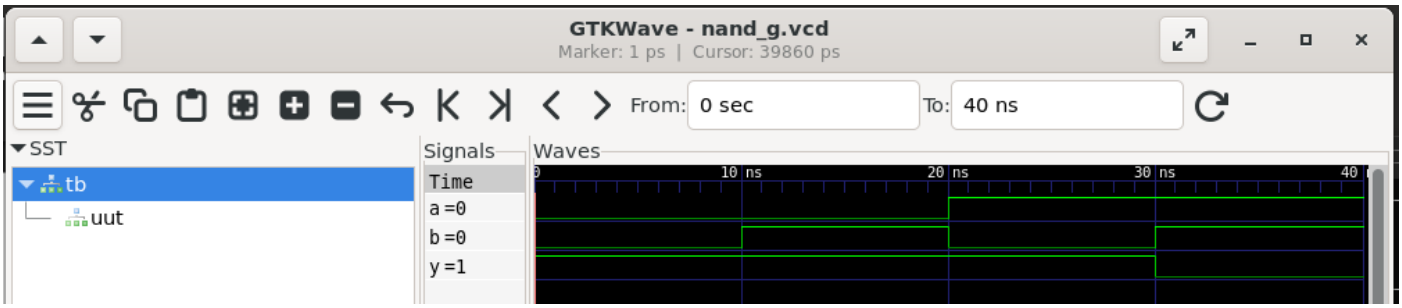
Nand Gate

Verilog Code:

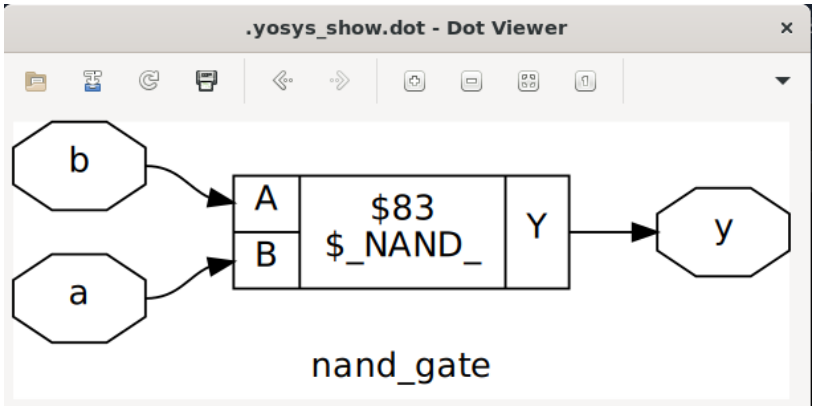
```
module nand_gate( input a,
 input b,
 output y);

assign y = ~(a&b);
endmodule
```

Testbench:

```
`timescale 1ns/1ps
module tb;
  reg a, b;
  wire y;

  nand_gate uut(.a(a), .b(b), .y(y));

  initial begin
    $dumpfile("nand_g.vcd");
    $dumpvars(0, tb);

    a = 0; b = 0; #10;
    a = 0; b = 1; #10;
    a = 1; b = 0; #10;
    a = 1; b = 1; #10;

    $finish;
  end
endmodule
```



Simulation and Synthesis output:



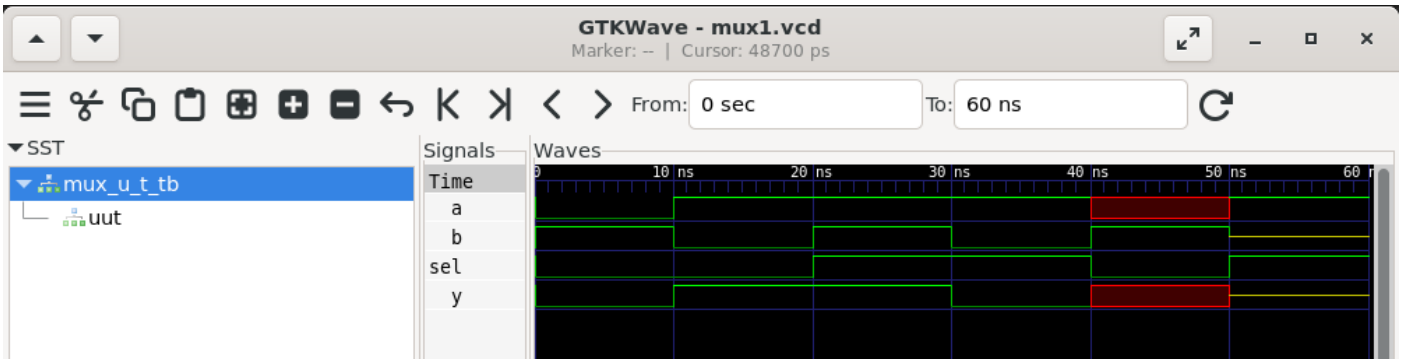
2.26. Printing statistics.

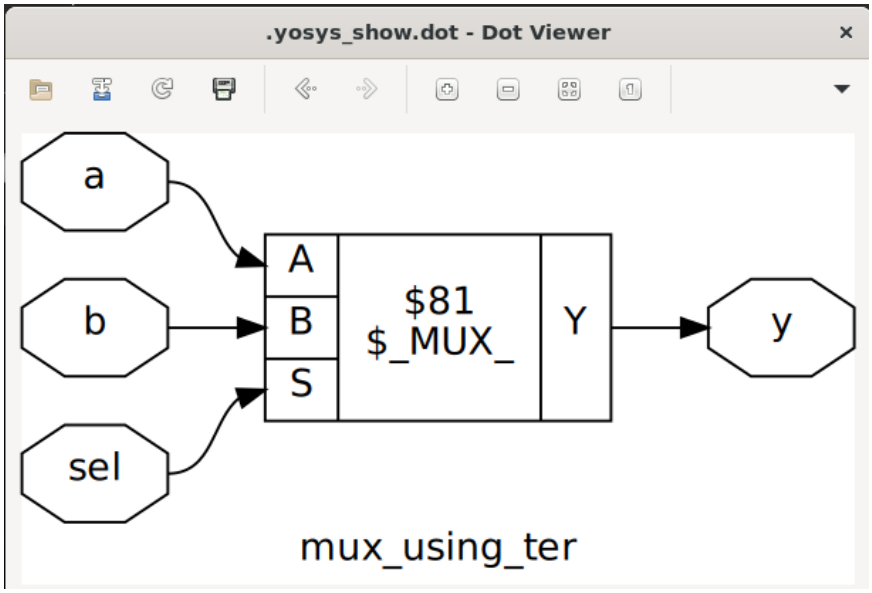
=== nand_gate ===

```
   Number of wires:                 3
   Number of wire bits:             3
   Number of public wires:          3
   Number of public wire bits:      3
   Number of memories:              0
   Number of memory bits:           0
   Number of processes:             0
   Number of cells:                 1
     $_NAND_                        1
```

2:1 MUX using ternary operators

```
module mux_using_ter( input a,
 input b,
 input sel,
 output y);

assign y = sel? b : a;

endmodule

`timescale 1ns/1ps
module mux_u_t_tb;
 reg a, b, sel;
 wire y;

 mux_using_ter uut(.a(a), .b(b), .sel(sel), .y(y));

initial begin
 $dumpfile("mux1.vcd");
 $dumpvars(0, mux_u_t_tb);

 sel = 0; a = 0; b = 1; #10
 sel = 0; a = 1; b = 0; #10
 sel = 1; a = 1; b = 1; #10
 sel = 1; a = 1; b = 0; #10
 sel = 0; a = 1'bx; b = 1; #10
 sel = 1; a = 1; b = 1'bz; #10

 $finish;
end
endmodule
```





2.26. Printing statistics.

```
=== mux_using_ter ===

   Number of wires:                 4
   Number of wire bits:             4
   Number of public wires:          4
   Number of public wire bits:      4
   Number of memories:              0
   Number of memory bits:           0
   Number of processes:             0
   Number of cells:                 1
     $_MUX_                         1
```

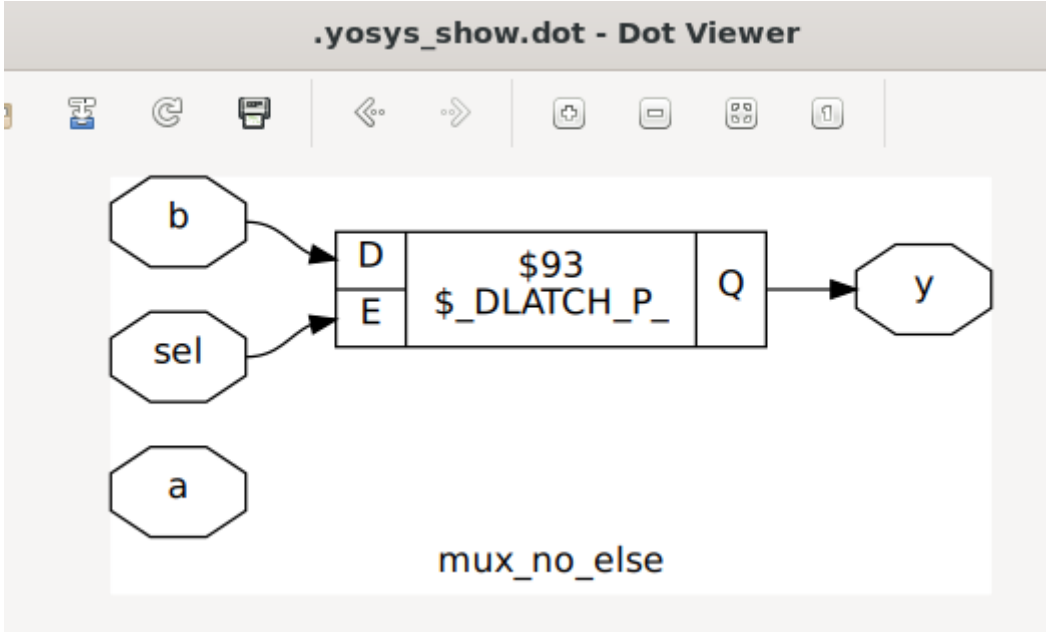
Similar synthesis for mux using case and if else

```
module mux_using_case( input a,
 input b,
 input sel,
 output reg y);
always @(*) begin

case (sel)
 1'b0: y = a;
 1'b1: y = b;

endcase
end
endmodule

module mux_using_if_else( input a,
 input b,
 input sel,
 output reg y);

always @(*) begin
 if(sel)
   y = b;
 else
   y = a;
end
```

endmodule

Important Notice: When no else is present, there is a latch effect happening, also the synthesizer doesn't know, what to do when select is 0, and has kept a as floating

```
module mux_no_else(input a, b, sel, output reg y);
   always @(*) begin
      if (sel) y = b;
      // no else
   end
endmodule
```



2.26. Printing statistics.

```
=== mux_no_else ===

   Number of wires:                 4
   Number of wire bits:             4
   Number of public wires:          4
   Number of public wire bits:      4
   Number of memories:              0
   Number of memory bits:           0
   Number of processes:             0
   Number of cells:                 1
     $_DLATCH_P_                    1
```